

Deploying OpenStack with Kolla-Ansible

A comprehensive, step-by-step guide for DevOps engineers and cloud administrators – from bare-metal node preparation through full OpenStack cluster deployment and core service configuration.

OPERATIONS GUIDE

OPENSTACK · KOLLA-ANSIBLE



Deployment Overview

This guide is organized into three sequential phases. Each phase must be completed in order before proceeding to the next. Together they take you from raw infrastructure to a fully operational OpenStack cloud.



Phase 1

Environment Preparation – nodes, networking, Docker, and Python dependencies



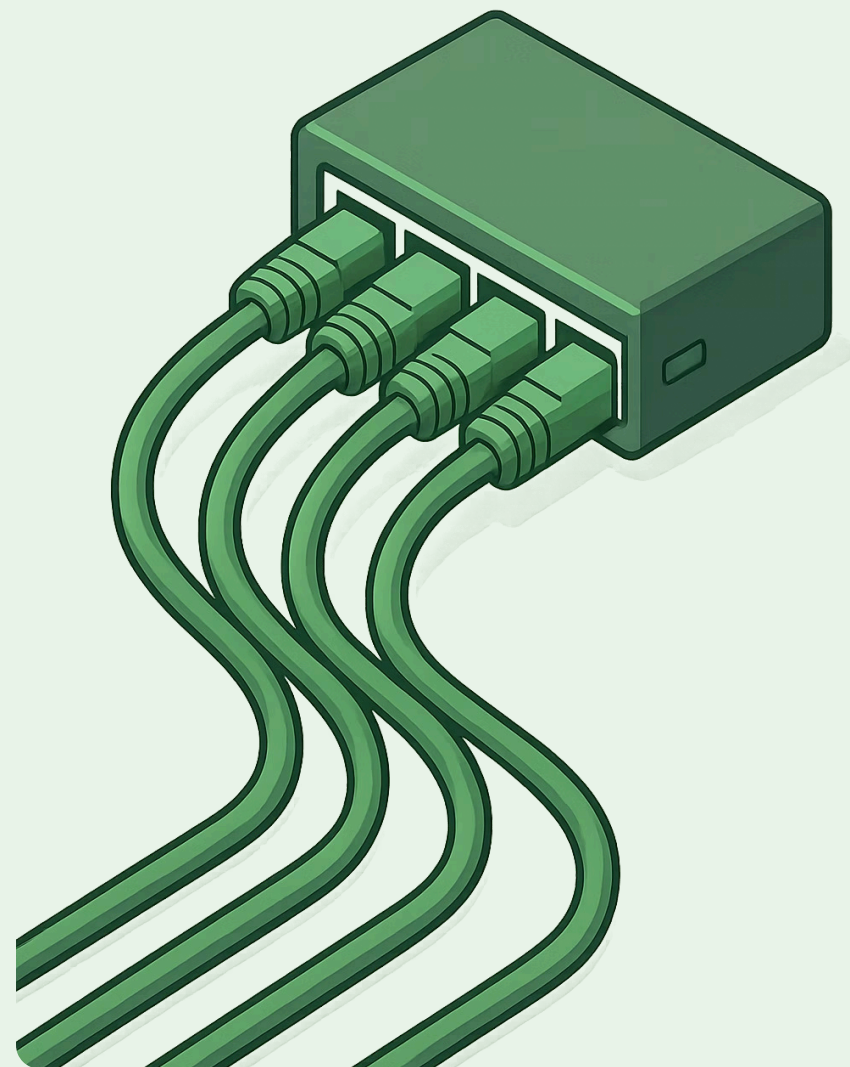
Phase 2

Kolla-Ansible Deployment – install, configure, bootstrap, pre-check, and deploy



Phase 3

Core Services Configuration – Keystone, Glance, Nova, Neutron, Horizon, and more



PHASE 1

Environment Preparation

Before any Kolla-Ansible commands can run, your infrastructure must be correctly configured. Controller and compute nodes need proper hostnames, SSH key-based access, and synchronized clocks. Networking prerequisites – including dedicated management and provider network interfaces – must be in place. Docker and all required Python packages must be installed at compatible versions.

Configuring Controller & Compute Nodes

Controller Node

- Set a fully qualified hostname (/etc/hostname)
- Configure static IP on the management interface
- Enable passwordless SSH from the deploy host
- Synchronize clocks via NTP (chrony recommended)
- Disable SELinux / AppArmor enforcement
- Open required firewall ports for API endpoints

Compute Nodes

- Repeat hostname and SSH key setup on each node
- Verify hardware virtualization: `egrep -c 'vmx|svm' /proc/cpuinfo`
- Load KVM kernel modules: `kvm_intel` or `kvm_amd`
- Confirm NTP synchronization across all nodes
- Ensure consistent user/group IDs across all hosts

- ❑ All nodes must have identical OS versions and kernel configurations to avoid compatibility issues during deployment.

Networking Prerequisites

Kolla-Ansible requires at least two network interfaces per node – a management network for API and internal traffic, and a dedicated provider (external) network interface that must remain unconfigured at the OS level so Neutron can manage it directly.

Management Network

Used for OpenStack API calls, inter-service communication, and Ansible SSH access. Assign a static IP on each node. This is the `network_interface` in `globals.yml`.

Provider / External Network

The raw interface handed to Neutron for floating IPs and external connectivity. **Do not assign an IP address** to this interface. Set it as `neutron_external_interface` in `globals.yml`.

DNS & Routing

Ensure all nodes can resolve each other by hostname. Configure `/etc/hosts` entries or a local DNS server. Verify outbound internet access for pulling container images from Docker Hub or a private registry.

Docker & Python Dependency Setup

Kolla-Ansible runs all OpenStack services inside Docker containers. Install Docker Engine on every target host and configure it to start on boot. On the deploy host, install the Kolla-Ansible Python package and its dependencies into an isolated virtual environment.

On All Target Hosts

- Install Docker Engine (20.10+ recommended)
- Add the `ansible` user to the `docker` group
- Enable and start the Docker daemon: `systemctl enable --now docker`
- Set `storage-driver` to `overlay2` in `/etc/docker/daemon.json`

On the Deploy Host

- Create a virtualenv: `python3 -m venv kolla-venv`
- Upgrade pip: `pip install -U pip`
- Install Ansible: `pip install ansible-core`
- Install Kolla-Ansible: `pip install kolla-ansible`
- Install Python Docker SDK: `pip install docker`



PHASE 2

Kolla-Ansible Deployment

With the environment ready, Kolla-Ansible orchestrates the full OpenStack deployment in a well-defined sequence: install and configure the tool, build the inventory, bootstrap target hosts, run pre-deployment checks, and finally execute the deploy.

From Install to Deploy: The Kolla-Ansible Workflow



1 • Install Kolla-Ansible

Install into a Python virtualenv. Copy the default `globals.yml` and `passwords.yml` from the package share directory to `/etc/kolla/`.



2 • Configure Inventory

Edit the multinode or all-in-one inventory file. Assign hostnames to `[control]`, `[network]`, `[compute]`, `[storage]`, and `[monitoring]` groups.



3 • Bootstrap Servers

Run `kolla-ansible bootstrap-servers` to install Docker, configure kernel parameters, and set up NTP on all target nodes automatically.



4 • Pre-checks

Execute `kolla-ansible prechecks` to validate connectivity, port availability, disk space, and configuration correctness before any containers are pulled.



5 • Deploy

Run `kolla-ansible deploy`. Ansible pulls container images, starts all services in dependency order, and configures inter-service credentials automatically.

Key `globals.yml` Settings

Before bootstrapping, review and set the following critical parameters in `/etc/kolla/globals.yml`. These values drive the entire deployment and cannot be easily changed post-install.

kolla_base_distro

Set to `ubuntu` or `centos`. Must match the OS of your target nodes.

openstack_release

Pin to a specific release tag (e.g. `2024.1`) for reproducible deployments.

network_interface

The management NIC name (e.g. `eth0`) used for API and internal traffic.

neutron_external_interface

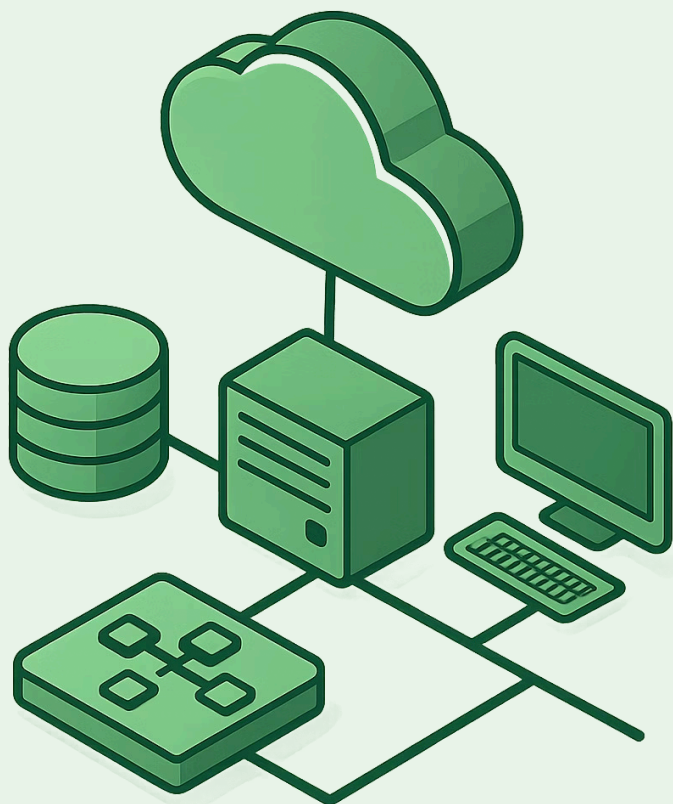
The raw NIC for Neutron's external network bridge. Must have no IP assigned.

kolla_internal_vip_address

A free IP on the management network used as the HA virtual IP for API endpoints.

enable_* flags

Toggle optional services like Cinder, Swift, Heat, or Ceilometer with boolean flags.



PHASE 3

Core Services Configuration

Once the cluster is deployed, each OpenStack service requires post-deployment tuning. The following cards walk through the nine core services – from identity and compute to messaging and caching – covering the key configuration decisions for a production-ready environment.

Identity, Image & Placement Services



Keystone — Identity Service

Configure the `admin` project, domain, and user via `kolla-ansible` post-deploy and the generated `admin-openrc.sh`. Set token expiry, enable Fernet tokens, and register all service endpoints in the catalog. Enforce TLS on public endpoints in production.



Glance — Image Service

Define the default image backend (`file` for single-node, `ceph` for HA). Upload a base image: `openstack image create`. Set `glance_store` policies and enable image conversion for format compatibility across compute drivers.



Placement — Resource Tracking

Placement tracks resource inventories (CPU, RAM, disk) for the Nova scheduler. Verify the Placement API is reachable with `openstack resource provider list`. Ensure `nova-compute` agents register their resource classes correctly on startup.

Compute & Networking Services

Nova — Compute Service

- Verify the hypervisor type: `nova-compute` auto-detects KVM
- Configure CPU and RAM allocation ratios in `nova.conf` for overcommit
- Set `live_migration_uri` for KVM live migration between compute nodes
- Tune the scheduler filters: enable `ComputeFilter`, `RamFilter`, `DiskFilter`
- Verify instance console access via `nova-novncproxy`

Neutron — Networking Service

- Choose ML2 plugin with `linuxbridge` or `openvswitch` mechanism driver
- Create an external flat network mapped to `neutron_external_interface`
- Define a provider subnet with floating IP allocation pool
- Create tenant networks using VXLAN or VLAN segmentation
- Configure security groups and default router for tenant access

❏ For production, enable Neutron HA with L3 agent redundancy and VRRP-based router failover.

Dashboard & Supporting Infrastructure

Horizon provides the web UI, while RabbitMQ, MariaDB, and Memcached form the critical messaging, persistence, and caching backbone that every OpenStack service depends on.



Horizon

Access the dashboard at `http://<VIP>/dashboard`. Set `ALLOWED_HOSTS` and configure TLS termination via HAProxy. Customize branding in `local_settings.py`. Verify login with the `admin` credentials from `admin-openrc.sh`.



RabbitMQ

All OpenStack services communicate asynchronously via RabbitMQ. Kolla deploys it as a cluster. Monitor queue depth and connection counts via the RabbitMQ management plugin at port `15672`. Tune `heartbeat` and `connection_timeout` for busy clusters.



MariaDB

Kolla deploys MariaDB as a Galera cluster for HA. Each service database is created automatically during deploy. Set `max_connections` and `innodb_buffer_pool_size` according to available RAM. Enable slow query logging for performance diagnosis.



Memcached

Used by Keystone for token caching and by other services to reduce database load. Kolla configures Memcached automatically. Verify it is reachable with `echo stats | nc <host> 11211`. Tune memory allocation via the `memcached_max_memory` variable in `globals.yml`.

Post-Deploy Validation Checklist

After all services are configured, run a structured smoke test to confirm the cluster is fully operational before handing it off or onboarding workloads.

Service Health Checks

- ✓ ~~Run kolla-ansible check — validates all containers are running~~
- Verify all API endpoints: openstack endpoint list
- Confirm Nova hypervisors: openstack hypervisor list
- Check Neutron agents: openstack network agent list
- Validate Placement: openstack resource provider list

Functional Smoke Tests

- Upload a test image to Glance
- Create a flavor and launch a test instance
- Assign a floating IP and verify SSH access
- Create and attach a Cinder volume (if enabled)
- Log in to Horizon and confirm dashboard renders

Key Takeaways & Next Steps

Structured Phases Win

Skipping environment preparation steps is the #1 cause of failed deployments. Validate each phase before advancing.

Pre-checks Save Time

kolla-ansible prechecks catches the majority of misconfigurations before any containers are pulled, saving significant troubleshooting time.

Services Are Interdependent

MariaDB, RabbitMQ, and Memcached must be healthy before API services will function. Always verify infrastructure services first.

Iterate with Confidence

Kolla-Ansible supports idempotent re-runs. Re-run deploy safely after configuration changes to apply updates without full rebuilds.

